Data Analytics in Cloud Computing

Exercise 1

Cloud Service Models Lab

IaaS (VM) • PaaS (Azure SQL Database) • Serverless (Azure Functions) • SaaS (PowerBI)

Introduction

In this lab you will compute the same set of basic analytics metrics using four different cloud service models. The goal is to experience the differences in responsibility boundaries (what you manage vs what the cloud provider manages) instead of only learning the definitions.

We will calculate 4 aggregate metrics in:

- **IaaS:** Azure Virtual Machine (Ubuntu) running a local Python script.
- **PaaS (classic):** Azure SQL Database (managed database) using SQL queries.
- **Serverless compute:** Azure Functions exposing an HTTP endpoint that returns metrics as JSON.
- **SaaS (optional demo):** A browser-based analytics/reporting application (Power BI Service or an equivalent BI SaaS).

Prerequisites

- An active Azure subscription and permission to create resources (VM, SQL Database, Function App).
- Recommended tools for Task 2: bcp utility
- Basic familiarity with SSH, Python, and SQL.

Files used in this lab

- EX_1/cloud_orders.csv
- EX_1/agg_metrics.py
- EX_1/azure_sql_create_table.sql
- EX_1/azure_sql_metrics.sql

Dataset

We will use the file cloud_orders.csv. It contains synthetic order data for a fictional cloud marketplace.

Columns

Column	Meaning
order_id	Order identifier (string).
order_date	Order date (YYYY-MM-DD).
region	Sales region (EMEA, NA, APAC, LATAM).
country	Customer country.
customer_id	Customer identifier (string).
customer_segment	Customer segment (SMB, Enterprise, Consumer).
product	Product category (Compute, Storage, Database, Analytics, AI).
channel	Sales channel (Web, Partner, Direct).
units	Quantity (integer).
unit_price_usd	List unit price (USD).
unit_net_usd	Net unit price after synthetic discounts (USD).
revenue_usd	Net revenue for the order line (units * unit_net_usd).

Metrics to calculate

For all tasks, calculate the same four metrics:

- total_orders = COUNT(*)
- total_revenue_usd = SUM(revenue_usd)
- avg_order_value_usd = AVG(revenue_usd)
- top_product_by_revenue = product with the highest SUM(revenue_usd)

Expected results (sanity check)

Metric	Expected value
total_orders	240
total_revenue_usd	123103.57
avg_order_value_usd	512.93
top_product_by_revenue	AI (44606.03)

Responsibility boundaries (quick reference)

Model	You manage	Provider manages	What you deploy
IaaS (VM)	OS updates, packages, runtime, your scripts, security hardening	Physical servers, virtualization, networking fabric, datacenter	Scripts/apps + config
PaaS (Azure SQL)	Schema, users, queries, data quality, permissions	Database engine patching, backups, HA, OS/hosts	DDL/DML + queries
Serverless (Functions)	Function code, app settings, auth settings, monitoring configuration	Server provisioning, auto-scale, host runtime	Function code
SaaS (PowerBI)	Dashboards/reports, user access, dataset refresh config	Application + platform + scaling + upgrades	Configuration (no code deployment)

Task 1 - IaaS: Run analytics on an Azure Virtual Machine

Learning objectives

- Provision a Linux VM in Azure.
- Connect via SSH.
- Download the dataset and helper script from your GitHub repository.
- Run the script to calculate the metrics.

Guide on how to create VM

Step 1: Create the VM

In the Azure portal create a Resource Group (for example: rg-service-models-lab). Then create a Linux Virtual Machine with these recommended settings:

- Image: Ubuntu Server 24.04 LTS
- Size: B2as_v2 (or another small size)
- Authentication: SSH key (recommended) or password
- Inbound ports: allow SSH (22) - restrict source to your IP if possible

Step 2: Connect to the VM (SSH)

Open the VM resource in the portal, click Connect -> SSH, and copy the suggested ssh command. You can run it from your terminal or from Azure Cloud Shell.

Step 3: Download files and run

On the VM, download the dataset and the helper script from your repo and run the script.

```
curl -L "https://raw.githubusercontent.com/MySlav/Data-Analytics-in-Cloud-Computing-
2025-2026/refs/heads/main/EX_1/cloud_orders.csv" -o cloud_orders.csv
curl -L "https://raw.githubusercontent.com/MySlav/Data-Analytics-in-Cloud-Computing-
2025-2026/refs/heads/main/EX_1/agg_metrics.py" -o agg_metrics.py
python3 agg_metrics.py cloud_orders.csv
```

Verify that the output matches the expected values in the sanity check table.

Task 2 - PaaS (classic): Azure SQL Database

Learning objectives

- Provision an Azure SQL Database (managed database).
- Create a table for the dataset.
- Import the CSV into the database.
- Run SQL queries to calculate the metrics.

How to create DB

Step 1: Create Azure SQL Database

In the Azure portal create a new SQL Database. Create a new SQL server when prompted. Recommended settings:

- Use the same Resource Group as the VM (rg-service-models-lab).
- Choose a low-cost compute option for the demo.
- Networking: Public endpoint; allow your client IP address.

Step 2: Create the table

Run the contents of your sql/azure_sql_create_table.sql file (or paste the SQL below) in query editor on azure:

```
-- azure_sql_create_table.sql
CREATE TABLE dbo.cloud_orders (
  order_id            varchar(20)  NOT NULL,
  order_date          date         NOT NULL,
  region              varchar(10)  NOT NULL,
  country             varchar(30)  NOT NULL,
  customer_id         varchar(10)  NOT NULL,
  customer_segment    varchar(20)  NOT NULL,
  product             varchar(20)  NOT NULL,
  channel             varchar(10)  NOT NULL,
  units               int          NOT NULL,
  unit_price_usd      decimal(10,2) NOT NULL,
  unit_net_usd        decimal(10,2) NOT NULL,
  revenue_usd         decimal(12,2) NOT NULL
);
```

Step 3: Import the CSV

Recommended: Use bcp. https://learn.microsoft.com/en-us/sql/tools/bcp-utility?view=sql-server-ver17&tabs=windows

- Install bcp utility
- Download cloud_orders.csv locally (from your GitHub repo).
- Open Powershell and import data using this command

```
bcp dbo.cloud_orders in "cloud_orders.csv"   -S tcp:<SERVER>.database.windows.net,1433 -d
<DBNAME> -U <USERNAME> -P <PASSWORD>   -c -t "," -r 0x0a -F 2
```

- Right-click your database -> Import Wizard (SQL Server Import extension).
- Select the CSV file and import into dbo.cloud_orders.

Step 4: Run the metric queries

Run your metric queries on Azure portal using query editor:

```
SELECT COUNT(*) AS total_orders,
       SUM(revenue_usd) AS total_revenue_usd,
       AVG(revenue_usd) AS avg_order_value_usd
FROM dbo.cloud_orders;

SELECT TOP 1 product,
       SUM(revenue_usd) AS revenue_usd
FROM dbo.cloud_orders
GROUP BY product
ORDER BY revenue_usd DESC;
```

Task 3 - Serverless compute: Azure Functions (HTTP endpoint)

Learning objectives

- Create a Function App.
- Store the dataset URL as an application setting.
- Create an HTTP-trigger function that downloads the CSV and returns JSON metrics.

Portal guide to create a Function App
Portal guide to create an HTTP endpoint
HTTP trigger concept

Step 1: Create a Function App

In the Azure portal create a new Function App with Python runtime (Consumption plan is OK for this lab). After creation, open the Function App resource.

Step 2: Add an app setting for the dataset URL

In Function App -> Settings -> Environment variables -> App settings, add:

- Name: DATASET_URL
- Value: <RAW_DATASET_URL>

Step 3: Create an HTTP trigger function and paste code

Create a new function (HTTP trigger). Set Authentication to Anonymous. In Code + Test, replace the code with:

```
import azure.functions as func
import csv, json, os
from collections import defaultdict
from urllib.request import urlopen
app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)

@app.route(route="metrics", methods=["GET"])
def metrics(req: func.HttpRequest) -> func.HttpResponse:
    data_url = os.environ.get("DATASET_URL")
    if not data_url:
        return func.HttpResponse("Missing DATASET_URL app setting", status_code=500)

    total_orders = 0
    total_revenue = 0.0
    rev_by_product = defaultdict(float)

    with urlopen(data_url) as resp:
        lines = resp.read().decode("utf-8").splitlines()
        r = csv.DictReader(lines)
        for row in r:
            total_orders += 1
            revenue = float(row["revenue_usd"])
            total_revenue += revenue
            rev_by_product[row["product"]] += revenue

    avg_order_value = round(total_revenue / total_orders, 2) if total_orders else 0.0
    top_product, top_product_rev = max(rev_by_product.items(), key=lambda x: x[1])

    out = {
        "total_orders": total_orders,
        "total_revenue_usd": round(total_revenue, 2),
        "avg_order_value_usd": avg_order_value,
        "top_product_by_revenue": {"product": top_product, "revenue_usd": round(top_product_rev, 2)}
    }
    return func.HttpResponse(json.dumps(out, indent=2), mimetype="application/json")
```

Step 4: Test the endpoint

Copy the Function URL from the portal and open it in your browser. It should return JSON with the same metrics.

Task 4 - SaaS: Visualize the dataset in a BI tool (Power BI Service, browser only)

Goal: Experience SaaS: you do not deploy code or manage infrastructure—only use and configure a web application.

Prerequisites

- Access to Power BI Service in a browser (app.powerbi.com).
- A workspace you can use (e.g., My workspace or the course workspace).
- The file cloud_orders.csv on your computer (or available to download).

Note: With a free license you can build reports in your own workspace, but sharing/collaboration may be limited.

Step 1: Import the CSV as a dataset (Semantic model)

1. Sign in to Power BI Service (app.powerbi.com).
2. Open your workspace (recommended: My workspace).
3. Click New item.
4. Under Store data, choose Semantic model.
5. Choose CSV.
6. Select cloud_orders.csv and click Import.
7. Wait until the semantic model (dataset) is created successfully.

Tip: If you do not see an obvious "upload CSV" option on the home page, always use the workspace path: Workspace → New item → Semantic model → CSV.

Step 2: Create a report from the dataset

- In the workspace, click the newly created Semantic model (dataset).
- Choose Create report (or Create → Report, then select your dataset).
- The report editor opens in the browser.

Step 3: Build the required visuals (4 metrics)

Create the following visuals on the report canvas:

1) Card — Total revenue

- Visual: Card
- Field: revenue_usd
- Aggregation: Sum

2) Card — Total orders

- Visual: Card
- Field: order_id
- Aggregation: Count (or Count (Distinct) if you want unique orders)

3) Card — Average order value

- Visual: Card
- Field: revenue_usd
- Aggregation: Average

4) Column/Bar chart — Revenue by product

- Visual: Clustered column chart (or bar chart)
- Axis: product
- Values: revenue_usd (Aggregation: Sum)

Optional (nice additions):

- Add a Slicer for region or customer_segment.
- Add another chart: Revenue by region (Axis: region, Values: Sum of revenue_usd).

Cleanup

To avoid unexpected charges, delete all resources created for this lab.

- Azure portal -> Resource groups -> rg-service-models-lab -> Delete resource group.